Projeto 2: Estoque de Insumos

> Material para os alunos · React Native (Expo SDK 54) + Firebase · parte do projeto-base de cadastro de alunos

1. Contexto

Sementes, adubos, defensivos e ração são comprados em grande quantidade e, sem controle, vencem no galpão ou acabam no meio da safra. Seu app será o controle de estoque da fazenda, avisando o que está acabando e o que está perto de vencer.

2. Objetivo

Construir um aplicativo mobile (rodando também no navegador) que faça o controle de **insumos**, com login, cadastro, listagem, alteração, exclusão e uma funcionalidade diferencial do tema. Você vai **partir do projeto-base** (cadastro de alunos) e adaptá-lo ao seu tema.

3. O que você vai reaproveitar do projeto-base

Arquivo	O que faz
`firebase.ts`	Inicializa o Firebase e exporta `auth` e `db`.
`components/frm-login.tsx`	Tela de login com `signInWithEmailAndPassword`.
`components/frm-cad-aluno.tsx`	Formulário que cadastra (`addDoc`) e altera (`updateDoc`) conforme o parâmetro `id`.
`components/lst-aluno.tsx`	Lista (`FlatList`) com botões Alterar e Excluir (`deleteDoc`).
`hooks/use-alunos.ts`	Hook que consulta a coleção (`getDocs`) e um documento (`getDoc`).
`hooks/use-focus-refresh.ts`	Recarrega os dados toda vez que a tela recebe foco.
`app/(tabs)/_layout.tsx`	Barra de abas com as cores do tema.
`app/(tabs)/index.tsx`	Tela inicial com botão Sair (`signOut`).

4. Modelo de dados (Firestore)

Coleção `insumos`

Campo	Tipo	Obrigatório	Descrição
`nome`	string	Sim	Nome do produto
`categoria`	string	Sim	semente, fertilizante, defensivo ou racao
`unidade`	string	Sim	kg, L, saco, dose…
`quantidade`	number	Sim	Quantidade atual em estoque

Campo	Tipo	Obrigatório	Descrição
`estoqueMinimo`	number	Sim	Abaixo disso o item entra em alerta
`validade`	string	Não	Data AAAA-MM-DD
`fornecedor`	string	Não	Nome do fornecedor

Subcoleção `insumos/{id}/movimentacoes`

Campo	Tipo	Descrição
`tipo`	string	entrada ou saida
`quantidade`	number	Quantidade movimentada
`data`	string	AAAA-MM-DD
`observacao`	string	Ex.: Aplicado no talhão 3

> Datas são guardadas como texto `AAAA-MM-DD`; números (peso, quantidade, horas…) devem ser gravados como `number`, não como texto.

5. Telas do aplicativo

1. Login
2. Início (com Sair)
3. Cadastro/edição de insumo
4. Lista de insumos
5. Tela de movimentação (entrada/saída)
6. Histórico de movimentações do insumo

6. Atividades

As atividades são sequenciais: cada uma depende da anterior. Total: **100 pontos**.

Atividade 1 – Preparação do ambiente (5 pontos)

O que fazer

- Clonar/copiar o projeto-base e rodar com `npx expo start`.
- Criar o **seu** projeto no console do Firebase, ativar **Authentication (e-mail/senha)** e **Firestore**.
- Trocar a configuração em `firebase.ts` pela do seu projeto.
- Cadastrar um usuário de teste no console do Firebase.
- Definir as regras do Firestore para aceitar apenas usuários autenticados (`request.auth != null`).

Entregável: Print do app rodando + print do console Firebase com o usuário e a coleção criados.

Critério principal: O app abre; o Firebase é o seu; regras não estão abertas para qualquer pessoa.

Atividade 2 – Autenticação (login e logout) (10 pontos)

O que fazer

- Adaptar a tela de login do modelo com a identidade do seu tema.
- Mostrar mensagem de erro com `Toast` quando usuário/senha forem inválidos.
- Implementar o botão **Sair** que encerra a sessão e volta ao login.

Entregável: Vídeo curto (até 1 min) ou prints: login correto, login errado, logout.

Critério principal: Login/erro/logout funcionam; o Sair volta para a tela de login.

Atividade 3 – Cadastro (insumo) (10 pontos)

O que fazer

- Criar o formulário com todos os campos obrigatórios do modelo de dados.
- Validar os campos obrigatórios e tipos (número, data) antes de gravar.
- Gravar com `addDoc` na coleção `insumos` e mostrar `Toast` de sucesso ou erro.
- Limpar o formulário após salvar.

Entregável: Prints do formulário preenchido e do documento criado no Firestore.

Critério principal: Dados chegam ao Firestore com os tipos certos (número como `number`).

Atividade 4 – Listagem (10 pontos)

O que fazer

- Criar um **hook** (`hooks/use-insumos.ts`) seguindo o modelo `use-alunos.ts`.
- Listar com `FlatList`, mostrando os principais campos em cartões.
- Recarregar a lista ao voltar para a tela (`useFocusRefresh`).
- Mostrar mensagem amigável quando a lista estiver vazia.

Entregável: Print da lista com pelo menos 5 registros.

Critério principal: A lista atualiza sozinha após cadastrar/alterar/excluir.

Atividade 5 – Alteração e exclusão (15 pontos)

O que fazer

- Botão **Alterar** abre o formulário com os dados carregados (`getDoc` + parâmetro `id`).
- No modo edição, o botão muda para **Alterar** e grava com `updateDoc`.
- Botão **Excluir** com confirmação (`Alert`) antes de chamar `deleteDoc`.
- Após salvar, o formulário volta ao modo cadastro.

Entregável: Vídeo curto ou prints do fluxo: alterar um registro e excluir outro.

Critério principal: Edição não cria registro duplicado; exclusão pede confirmação.

Atividade 6 – Identidade visual (5 pontos)

O que fazer

- Aplicar a paleta do projeto-base (ou criar uma nova, justificando) em todas as telas.
- Usar nomes de abas claros e ícones coerentes com o tema.
- Remover do app todo conteúdo de exemplo do template do Expo.

Entregável: Prints de todas as telas.

Critério principal: Visual consistente entre telas; nenhuma tela de template sobrando.

Atividade 7 – Funcionalidade diferencial: Movimentações de estoque e alertas (20 pontos)

O que fazer

- Tela para registrar entrada ou saída, gravando em `movimentacoes` e atualizando `quantidade` do insumo.
- Bloquear saída maior que a quantidade em estoque, com mensagem (`Toast`) clara.
- Na lista, destacar insumos com `quantidade <= estoqueMinimo` e os que vencem em até 30 dias.
- Filtro da lista por `categoria` e busca por nome.
- Tela inicial com o total de itens em alerta.

Entregável: Vídeo curto (até 2 min) mostrando a funcionalidade e prints do Firestore.

Critério principal: Todos os itens funcionam e os dados ficam consistentes entre as coleções.

Atividade 8 – Documentação e repositório (10 pontos)

O que fazer

- Escrever o `README.md` do seu projeto: objetivo, como rodar, telas e modelo de dados.
- Comentar o código nos pontos principais (hooks e consultas ao Firestore).
- Subir o código em um repositório Git com commits ao longo do projeto.

Entregável: Link do repositório + README + apresentação.

Critério principal: Qualquer colega consegue rodar o projeto seguindo o seu README.

Atividade 9 – Apresentação em grupo (até 15 minutos) (15 pontos)

O que fazer

- Montar os slides seguindo o **roteiro de apresentação** descrito no fim deste documento.
- Mostrar o app funcionando (demonstração ao vivo ou vídeo de reserva).
- Explicar a ferramenta (Expo/React Native e Firebase) e os **desafios** que o grupo enfrentou com o framework e com a tecnologia.
- Dividir a fala entre **todos** os integrantes e respeitar o tempo de 15 minutos.

Entregável: Slides (PDF) entregues antes da apresentação + apresentação oral.

Critério principal: Cumpre o roteiro, o tempo e mostra domínio do que foi construído (cada integrante responde perguntas).

Desafios extras (opcionais, +5 pontos cada, até 10)

- Ordenar por validade
- Relatório de saídas do mês

7. Cronograma sugerido

Período	Atividades
Semana 1	Atividades 1 e 2
Semana 2	Atividades 3 e 4
Semana 3	Atividades 5 e 6
Semana 4	Atividade 7
Semana 5	Atividades 8 e 9 (documentação e apresentações)

8. Avaliação

Atividade	Pontos
1 – Preparação do ambiente	5
2 – Autenticação (login e logout)	10
3 – Cadastro (insumo)	10
4 – Listagem	10
5 – Alteração e exclusão	15
6 – Identidade visual	5
7 – Funcionalidade diferencial: Movimentações de estoque e alertas	20
8 – Documentação e repositório	10
9 – Apresentação em grupo (até 15 minutos)	15
Total	**100**

O que descontamos: chaves/senhas pessoais no repositório público, código copiado sem entender, telas do template do Expo sobrando, app que não abre.

9. Dicas importantes

- **Leia a documentação da versão certa**: este projeto usa Expo SDK 54 (`https://docs.expo.dev/versions/v54.0.0/`). Tutoriais antigos podem ensinar APIs que mudaram.
- Faça commits pequenos e frequentes: um por atividade, no mínimo.
- Teste no navegador (`w` no terminal do Expo) e no celular (Expo Go) quando puder.
- Se o Firestore recusar a gravação, veja o código do erro no `Toast` e revise as regras de segurança.
- Paleta de cores sugerida (inspirada no UniSenai): `#1a3a8f` azul principal, `#002d66` azul escuro (texto), `#F7941D` laranja de destaque (botões), `#f5f7fc` fundo claro, `#e85d04` laranja-escuro (ações destrutivas).

10. Apresentação final e roteiro (até 15 minutos por grupo)

Cada grupo apresentará o projeto para a turma. A apresentação vale **15 pontos** (Atividade 9). Todos os integrantes devem falar e responder perguntas.

Roteiro sugerido

#	Tópico	Tempo	O que abordar
1	Abertura e problema	1 min	Nome do grupo, tema escolhido e o problema real que o app resolve.
2	Demonstração do app	3 min	Fluxo completo: login, cadastro, lista, alteração, exclusão e a funcionalidade diferencial. Tenha um vídeo de reserva.
3	A ferramenta: Expo e React Native	2 min	O que é React Native e o que o Expo acrescenta (Expo Go, Expo Router, rodar no celular e no navegador). Por que usar.

#	Tópico	Tempo	O que abordar
4	Firebase: Authentication e Firestore	2 min	Como funciona o login por e-mail/senha, coleções e subcoleções, e as regras de segurança. Mostre o console.
5	Arquitetura do projeto	2 min	Organização de pastas, componentes, hooks (ex.: `use-alunos`) e o caminho de um dado: tela → hook → Firestore → tela.
6	Desafios com o framework	2 min	Problemas com React Native/Expo/Expo Router e como foram resolvidos (veja as sugestões do seu tema).
7	Desafios com a tecnologia	2 min	Problemas com Firebase/Firestore/TypeScript e como foram resolvidos (veja as sugestões do seu tema).
8	Aprendizados e próximos passos	1 min	O que o grupo aprendeu, o que faria diferente e o que melhoraria no app.
	Total	**15 min**	

Desafios sugeridos para o seu tema

Escolha os que o seu grupo realmente enfrentou e conte **o problema, o que tentaram e a solução**.

Framework (React Native / Expo / Expo Router)

- Formulário de movimentação com campos numéricos e validação antes de salvar
- Filtro e busca na `FlatList` sem perder desempenho

Tecnologia (Firebase / Firestore / TypeScript)

- Manter `quantidade` do insumo e as `movimentacoes` consistentes (pensar em atualização atômica/transação)
- Impedir saída maior que o estoque e tratar o erro para o usuário

> Além dos sugeridos, vale citar os desafios comuns: rotas com o mesmo caminho (`/` do login e da tela inicial), loop de renderização por callback não memorizado e números gravados como texto.

Checklist antes de apresentar

- Slides prontos (cerca de 1 por minuto) e enviados em PDF.
- App rodando, com dados de exemplo já cadastrados.
- Vídeo de reserva caso a internet ou o Firebase falhe.
- Falas divididas entre todos os integrantes; tempo ensaiado.
- Chaves e senhas pessoais **fora** dos slides.

Avaliação da apresentação

Critério	Pontos
Conteúdo técnico (ferramenta e arquitetura corretas)	5
Desafios enfrentados e soluções (framework e tecnologia)	4
Demonstração funcionando	3
Clareza, divisão da fala e respeito ao tempo	3

Critério	Pontos
Total	**15**

11. Referências

Expo

- Documentação do Expo SDK 54 (versão do projeto)
- Expo Router – introdução
- Expo Router – abas (Tabs)
- Usando Firebase com Expo

Firebase

- Firestore – visão geral
- Firestore – adicionar dados (addDoc, setDoc, updateDoc)
- Firestore – ler dados (getDoc, getDocs)
- Firestore – excluir dados (deleteDoc)
- Firestore – consultas simples e compostas (where, orderBy)
- Firestore – subcoleções e modelo de dados
- Firestore – regras de segurança
- Authentication – e-mail e senha na Web

React e React Native

- React – useState
- React – useCallback
- React – criando seus próprios Hooks
- React Native – FlatList
- React Native – TextInput
- React Native – StyleSheet
- react-native-toast-message

TypeScript

- TypeScript Handbook